

[블록체인 기반 티켓 재판매 시스템]

프로젝트 관리 계획서

- 곽도혁 -

<https://github.com/sunnyside-up0102/blockchain-ticketing-system/tree/main>

문서번호 [곽도혁]프로젝트관리계획서_260410_Doc-

: 002

소속 : 한국항공대학교

팀명 : Block_Ticketing

팀원 : 곽도혁

교수 : 최차봉 교수님

제/개정 이력

[illegible]

목 차

1. 서론	1
1.1 문서의 목적 및 범위	1
1.2 프로젝트 개요	1
1.3 프로젝트 목표	2
1.4 용어 정의	2
1.5 참조 문서	2
2. 개발 계획	3
2.1 개발 절차 모형	3
2.2 WBS(Work Breakdown Structure)	3
2.3 개발 내용	4
2.4 개발 일정	5
3. 팀 구성	6
3.1 팀 구조	6
3.2 역할 및 책임	6
4. 품질 관리	7

4.1 팀 미팅 계획	7
4.2 변경 사항 관리 방법	7
4.3 산출물 관리 방법	8
5. 개발 환경	9
5.1 하드웨어 환경	9
5.2 소프트웨어 환경	9
5.3 기타 개발 환경	9
6. 산출물	10
6.1 산출물 정의	10
6.2 산출물 작성일 및 담당자	10
6.3 기타 제약사항	11
7. 기타 사항	11
7.1 리스크 관리	11
8. 참고문헌 및 부록	11

1. 서 론

1.1 문서 목적 및 범위

본 문서는 블록체인 기반 티켓 재판매 시스템 개발을 위한 프로젝트 관리 계획서를 정의하는 것을 목적으로 한다.

본 문서는 다음과 같은 내용을 포함한다.

- 프로젝트 개요 및 목표
- 개발 계획 및 일정
- 팀 구성 및 역할
- 품질 관리 및 운영 계획
- 개발 환경 및 산출물 정의

1.2 프로젝트 개요

1.2.1 프로젝트 정의

본 프로젝트는 블록체인을 활용하여 티켓 거래의 투명성과 신뢰성을 확보하는 재판매 시스템을 개발하는 것을 목표로 한다.

기존 티켓 거래 시스템은 중앙화 구조로 인해 위조 및 압표 문제가 발생할 수 있으나, 본 시스템은 NFT 기반 티켓과 스마트 계약을 활용하여 이러한 문제를 해결하고자 한다.

1.2.2 주요 기능 설명

ID	설명
F01	티켓 발행 및 NFT화: 공연 주최자가 티켓 정보를 블록체인에 기록하고 NFT 형태로 소유 자산을 생성함
F02	소유권 이전 및 거래: 사용자 간 티켓 구매 시 소유권 정보가 블록체인 상에서 투명하게 이전됨
F03	재판매 관리: 재판매 과정의 모든 기록을 블록체인에 저장하여 거래의 투명성과 신뢰성을 확보함
F04	QR 코드 기반 인증: 입장 시 블록체인에 저장된 티켓 정보를 조회하여 QR 코드로 진위 여부를 실시간 검증함
F05	거래 이력 조회: 사용자가 티켓의 최초 발행부터 현재 소유자까지의 모든 유통 과정을 투명하게 조회함

1.3 프로젝트 목표

본 프로젝트의 목표는 블록체인 기술을 활용하여 기존 티켓 재판매 시장에서 발생하는 위조, 암표, 거래 불투명성 문제를 해결하는 것이다. 이를 위해 NFT 기반 티켓 발행 및 관리 시스템을 구축하고, 스마트 컨트랙트를 통해 거래 과정을 자동화하여 신뢰성을 확보하고자 한다.

또한 사용자가 직관적으로 시스템을 이용할 수 있도록 UI/UX를 고려한 환경을 제공하며, 거래 이력을 투명하게 확인할 수 있는 기능을 통해 사용자 신뢰도를 향상시키는 것을 목표로 한다. 나아가 본 시스템은 향후 확장 가능한 구조로 설계하여 다양한 공연 및 이벤트에 적용될 수 있도록 하는 것을 지향한다.

1.4 용어 정의

본 문서의 이해를 돕기 위해 사용된 모든 용어 및 약어를 설명하고 정의합니다.

용어	설명
블록체인	중앙 서버 없이 여러 노드가 데이터를 분산 저장하는 기술로, 데이터 위변조가 어렵고 투명성이 높은 특징을 가진다.
NFT (Non-Fungible Token)	대체 불가능한 토큰으로, 각 자산이 고유한 값을 가지며 디지털 소유권을 증명하는 데 사용된다.

스마트 컨트랙트	특정 조건이 충족되면 자동으로 실행되는 프로그램으로, 계약의 신뢰성과 자동화를 보장한다.
트랜잭션	블록체인 네트워크에서 발생하는 거래 단위로, 모든 기록은 블록에 저장된다.

1.4 참조 문서

시스템 정의서
형상관리 계획서

2. 개발 계획

2.1 개발 절차 모형

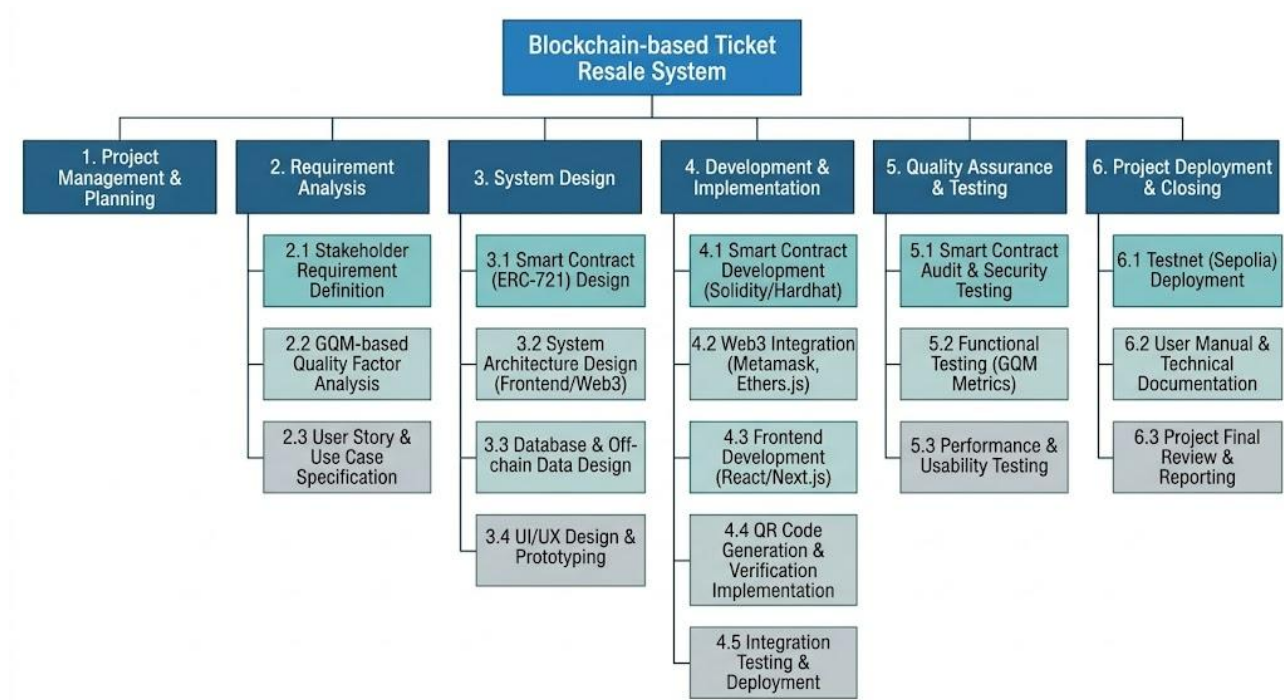
본 프로젝트는 요구사항의 불확실성과 기술적 복잡성을 고려하여 애자일 기반의 Scrum 개발 모델을 적용한다.

초기 단계에서는 요구사항이 명확하게 정의되기 어려우며, 블록체인 기술을 적용하는 과정에서 기능 수정 및 추가가 빈번하게 발생할 가능성이 높다. 따라서 전통적인 폭포수 모델보다는 반복적이고 점진적인 개발이 가능한 애자일 방식이 적합하다.

Scrum 모델에서는 Product Backlog를 기반으로 개발할 기능을 우선순위에 따라 관리하며, Sprint 단위로 기능을 개발한다. 각 Sprint는 1~2주 단위로 구성되며, Sprint Planning을 통해 개발 목표를 설정하고, Daily Scrum을 통해 진행 상황을 공유한다.

또한 Sprint 종료 시점에는 Sprint Review를 통해 결과물을 검토하고, Sprint Retrospective를 통해 개발 과정에서의 문제점과 개선사항을 도출한다. 이러한 반복 과정을 통해 프로젝트의 품질을 지속적으로 향상시키고, 변화하는 요구사항에 유연하게 대응할 수 있도록 한다.

2.2 WBS(Work Breakdown Structure)



2.3 개발 활동

개발 활동	설명
프로젝트 요약	프로젝트의 핵심 개념과 범위를 정의한 시스템 정의서 작성
프로젝트 계획	일정, 자원, 품질 관리 및 리스크 관리 계획을 포함한 프로젝트 관리 계획서 작성
요구사항 정의 및 분석	기능적/비기능적 요구사항을 도출하고 GQM 기반의 품질 요소를 분석하여 정의서 및 분석서 작성
Product Backlog	도출된 요구사항(티켓 발행, NFT 거래 등)을 우선순위에 따라 나열하여 관리
Sprint Planning Meeting	각 스프린트(2주 단위) 시작 전, 백로그에서 개발할 항목을 선정하고 세부 계획 수립
Daily Scrum Meeting	매일 정해진 시간에 진행 상황, 장애 요인, 당일 계획을 공유하여 팀 내 의사소통 최적화
Sprint	2주 단위의 반복 주기를 통해 스마트 컨트랙트 및 Web3 기능을 실제 구현하고 피드백 반영

Sprint Retrospective	스프린트 종료 후 프로세스상의 잘된 점과 개선할 점을 도출하여 다음 스프린트에 반영
Sprint Review	스프린트 결과물(실행 가능한 소프트웨어)을 검토하고 문서화하여 품질 확인
프로젝트 종료	최종 산출물 정리, 테스트 결과 보고서 작성 및 프로젝트 완료 보고

2.4 개발 일정

활동	4	5	6	7	8	9	10	11	12	1	2	3
프로젝트 요약												
프로젝트 계획												
요구사항 정의 및 분석												
Product Backlog												
Sprint Planning Meeting												
Sprint 1												
Sprint 2												
Sprint 3												
Sprint 4												
Sprint 5												
Sprint 6												
Sprint 7												
Sprint 8												
Sprint 9												

Sprint 10												
Daily Scrum Meeting												
Sprint Retrospective												
Sprint Review												
프로젝트 종료												

3. 팀 구성

3.1 팀 구조

팀장의 역할은 있지만 모든 멤버가 상호 협력하고 서로의 개발 활동을 확인하며 진행하는 수평적 팀 구조를 채택하였다.

3.2 역할 및 책임

이름	역할	책임 (공동 / 개별)	
곽도혁	팀장	문서관리 자료조사	프로젝트 총괄 및 일정 관리, 시스템 아키텍처 설계, 스마트 컨트랙트 개발
곽씨니	팀원		비즈니스 요구사항 분석, 백엔드 API 구현, 데이터베이스 모델링
곽제니	팀원		프론트엔드 UI/UX 기획 및 구현, 클라이언트 측 Web3 지갑 연동

(*과제용 문서에서는 팀원들은 모두 가상의 인물입니다.)

4. 품질 관리

4.1 팀 미팅 계획

본 프로젝트는 Scrum 기반으로 개발을 진행하며, 원활한 의사소통과 프로젝트 진행 관리를 위해 다음과 같은 미팅을 수행한다.

(1) Sprint Planning Meeting

- 팀 미팅 시간 : 매주 월요일 18시
- 토의 내용 : Sprint 목표와 Sprint Backlog를 계획
- 미팅 참석자 : 팀원, 팀장

(2) Daily Scrum Meeting

- 팀 미팅 시간 : 매일 오전 10시에 30분간 진행
- 토의 내용 : 전날 수행한 작업, 오늘 수행할 작업, 진행 중 발생한 문제 및 장애 요소
- 미팅 참석자 : 팀원, 팀장

(3) Sprint Review

- 팀 미팅 시간 : Sprint 종료 시점
- 토의 내용 : Sprint 기간 동안 개발된 기능 검토, 결과물 확인 및 개선 사항 도출
- 미팅 참석자 : 팀원, 팀장

(4) Sprint Retrospective

- 팀 미팅 시간 : Sprint 종료 시점
- 토의 내용 : Sprint 기간 동안 팀 내부에서 발생한 일들을 회고하며 개선할 것, 잘한 것, 새로 시작할 것을 도출

(5) 비정기 회의

- 프로젝트 진행에 문제가 발생하여 회의가 필요한 경우에 진행된다.
- 비정기 회의는 팀장 또는 지도교수 또는 멘토에 의해 요청될 수 있다.
- 미팅 참석자 : 팀원, 팀장, (지도교수, 멘토)

4.2 변경사항 관리 방법

본 프로젝트는 개발 과정에서 발생하는 요구사항 변경 및 기능 수정 사항을 체계적으로 관리하기 위해

다음과 같은 변경사항 관리 방법을 적용한다.

(1) 변경사항 관리 원칙

- 모든 변경사항은 사전에 정의된 절차에 따라 기록 및 관리한다.
- 변경 전/후 내용을 명확하게 비교할 수 있도록 문서화한다.
- 주요 기능 변경 시 전체 시스템에 미치는 영향을 고려하여 반영한다.

(2) 변경사항 관리 절차

1. 변경 요구 발생
→ 기능 수정, 추가, 삭제 등의 요구 발생
2. 영향 분석
→ 해당 변경이 시스템에 미치는 영향 분석
3. 변경 승인
→ 개인 프로젝트이므로 자체 판단 후 승인
4. 변경 수행
→ 코드 및 문서 수정
5. 변경 기록
→ 변경 내용을 GitHub 및 문서에 기록

(3) 버전 관리 방법

- GitHub 를 활용하여 소스코드 및 문서를 관리한다.
- 변경사항 발생 시 commit 을 통해 이력을 기록한다.
- 버전 관리는 다음과 같은 규칙을 따른다.

버전 형식: vMajor.Minor.Patch

- **Major (주 버전):** 전체 구조 변경, 큰 기능 추가
- **Minor (부 버전):** 기능 추가 또는 개선

- **Patch (수정 버전):** 버그 수정 및 간단한 변경
-

(4) 변경사항 기록 방법

- 모든 변경사항은 commit message 를 통해 기록한다.
- 주요 변경사항은 CHANGELOG.md 파일에 정리한다.
- 변경 이력은 추후 추적 가능하도록 유지한다.

4.3 산출물 관리 방법

(1) 산출물에 대한 명명(Naming) 방법

- 소프트웨어 : 버전_해당 버전의 산출 일자
(ex. v1_2019_10_11.exe)
- 문서 : [팀명]문서명_YYMMDD_Doc-해당 문서의 버전
(ex. [김검정]프로젝트관리계획서_191025_Doc-001.hwp)
- 그 외 파일 : 해당 파일의 목적이 명확하게 들어나는 이름
(ex. 191025_video.h264)

(2) 산출물의 버전 관리 방법 및 버전관리 담당자

- 소프트웨어 : 파일명 맨 앞에 버전을 기재한다. (담당자: 팀원 전체)
- 문서 : 파일명 맨 뒤에 날짜를 기재하고, 문서 내에 제/개정 이력을 수정한다. (담당자 : 김두회)
- 그 외 파일 : 파일명 앞에 날짜가 붙으며 필요할 경우 1,2 등의 숫자를 추가하여 기재한다.
(담당자 : 전체)

(3) 산출물의 저장 방법

- 소프트웨어 : Github
- 문서 : 하드웨어 및 팀원 공유 클라우드서버
- 그 외 파일 : 하드웨어 및 팀원 공유 클라우드

5. 개발 환경

5.1 하드웨어 개발 환경

구분	spec
개발용 PC	Intel i5 8세대 이상 CPU, RAM 16GB, SSD 256GB
테스트 기기	모바일 기기 (QR 코드 인증 및 Web3 지갑 연동 테스트용)

5.2 소프트웨어 개발 환경

구분	type	spec
운영체제	OS	Window 10 이상
개발도구	IDE	VS Code, Hardhat (Smart Contract Development)
언어/프레임워크	Stack	React, Next.js, Solidity
데이터베이스	DB	MySQL, Ethereum Testnet

5.3 기타 개발 환경

해당없음

6. 산출물

6.1 산출물 정의

형식 : [팀명]문서명_YYMMDD_Doc-해당 문서의 버전

ex)

- [곽도혁]시스템정의서_YYMMDD_Doc-해당 문서의 버전
- [곽도혁]프로젝트관리계획서_YYMMDD_Doc-해당 문서의 버전
- [곽도혁]요구사항정의서_YYMMDD_Doc-해당 문서의 버전
- [곽도혁]요구사항분석서_YYMMDD_Doc-해당 문서의 버전
- [곽도혁]소프트웨어설계서_YYMMDD_Doc-해당 문서의 버전

6.2 산출물 작성일 및 담당자

산출물	최종 작성일	최종 검토자	작성 책임자
[곽도혁]시스템정의서	2026-03-11	곽도혁	곽도혁
[곽도혁]프로젝트관리계획서	2024-04-10	곽도혁	곽도혁
[곽도혁]요구사항정의서			
[곽도혁]요구사항분석서			
[곽도혁]소프트웨어설계서			

6.3 기타 제약사항

해당사항 없음

7. 기타 사항

7.1 리스크 관리

No	Risk	Critical	Occur	Mitigation Plan	Alternative	Respon
1	컨트랙트 보안 취약점	High	Low	개발 단계에서 준비된 인스펙션(동료 검토) 양식지를 활용하여 정기적인 코드 리뷰를 수행	전문 보안 감사 도구(Mythril, Slither 등)를 도입하여 취약점을 전수 조사	곽도혁
2	이더리움 가스비 폭등	Medium	High	스마트 컨트랙트 코드 최적화를 통해 연산량을 최소화하고 가스 소모를 줄임	가스비가 저렴하고 전송 속도가 빠른 폴리곤(Polygon) 등 레이어 2(L2) 네트워크 도입을 검토함	곽도혁

8. 참고문헌 및 부록

-부록 1 : 시스템정의서 ([곽도혁]시스템정의서_260311_Doc-002)